

Product Requirements Document: IntentLock Correction Specification

Project: INTENTLOCK

Product: AI-Commerce Transaction Safety Gateway

Program: Razorpay Buildathon — Track 01: AI Growth & Agentic Commerce

Document status: Standalone implementation correction specification

Source of truth: The uploaded `Product_Requirements_Document_IntentLock.docx` and the attached correction requirements

Implementation rule: Build only the explicitly approved phase, stop, report results, and wait before proceeding

Critical correction: The current preview, “PERSONAL FOCUS CONSOLE,” is the wrong application. It must not be continued, extended, or treated as the intended product.

1. Executive Product Definition

IntentLock is a safety and control layer between an AI commerce agent and payment execution. It is not a productivity app, focus app, intention tracker, habit app, task manager, generic chatbot, generic shopping recommender, RL agent, custom ML model, fraud detection model, blockchain system, cryptocurrency system, payment gateway, microservice architecture, or production payment infrastructure.

The application must be recognizable immediately as:

Plain Text

```
INTENTLOCK
AI-Commerce Transaction Safety Gateway
```

The central principle is:

Plain Text

```
THE AI PROPOSES.
THE USER AUTHORIZES.
INTENTLOCK ENFORCES.
RAZORPAY EXECUTES.
```

IntentLock sits between an AI commerce agent and payment execution. The LLM interprets language and produces proposals, but it is never the source of financial authority.

IntentLock creates the authorization boundary, verifies live merchant state and the actual transaction, and makes the deterministic security decision. Only an **ALLOW** result may proceed to Razorpay Test Mode.

2. Current Implementation Correction

The current preview is labeled **PERSONAL FOCUS CONSOLE** and contains features such as “New focus” and “Set an intention.” This is the wrong application. The project is not a productivity app, focus app, intention tracker, habit app, or task manager.

The uploaded `Product_Requirements_Document_IntentLock.docx` is the source of truth. Any conflict between an implementation interpretation and that PRD must be resolved in favor of the PRD. The current preview is therefore a **FAILED IMPLEMENTATION** and must be corrected before continuing.

The first implementation task is to inspect the current project, identify the incorrect Personal Focus Console implementation, stop developing that product, reorient the application to IntentLock, preserve any generic project setup that remains useful, replace the incorrect product and UI, and build **only Phase 1**. The later phases must not be started automatically.

3. Required Product Workflow

The application must implement the following complete flow:

Plain Text

```
USER
↓
Natural-language purchase request
↓
LLM interprets the request
↓
Structured intent
↓
AI purchase proposal
↓
USER APPROVES
↓
IntentLock creates an authorization contract
↓
Current merchant state is verified
↓
Actual transaction request arrives
↓
```

```
IntentLock deterministically verifies the transaction
↓
ALLOW / REVIEW / BLOCK
↓
ONLY ALLOW can proceed to Razorpay TEST MODE
↓
Payment result
↓
Complete audit trail
```

The LLM is only a language and semantic layer. It must never be the source of financial authority, directly authorize payment, or directly execute payment. The deterministic IntentLock backend must remain capable of making the major security decisions even if the LLM is unavailable.

4. Product Behavior

A user may say:

“Buy me a laptop under ₹70,000.”

The LLM may convert this into:

Plain Text

```
Action: PURCHASE
Category: LAPTOP
Maximum amount: ₹70,000
Quantity: 1
```

The AI may propose:

Plain Text

```
Product: Lenovo X
Price: ₹69,999
Quantity: 1
```

After the user approves, IntentLock must create a machine-enforceable authorization containing, at minimum, the following information:

Authorization field	Requirement
Authorization ID	Identifies the authorization contract.
User ID	Identifies the authorizing user.

Agent ID	Identifies the AI commerce agent.
Merchant ID	Identifies the merchant.
Action	Defines the permitted action, such as purchase.
Product/item	Defines the authorized product or items.
Quantity	Defines the permitted quantity.
Maximum amount	Defines the spending ceiling.
Currency	Defines the currency, such as INR.
Allowed add-ons	Defines permitted additional commercial items.
Expiry	Defines when the authorization stops being valid.
Confirmation requirement	Records whether user confirmation is required.

IntentLock must then check both the current merchant state and the actual transaction.

5. Generic Verification Requirements

The verification engine must not hardcode rules for Lenovo, laptops, or any other specific product. Lenovo is only an example and demo product. The engine must work generically using IDs and transaction fields.

Every transaction must be evaluated across three comparable states:

Plain Text
AUTHORIZED vs CURRENT MERCHANT STATE vs ACTUAL TRANSACTION

The required checks are:

Verification invariant	Required comparison
Product	Authorized product versus actual product.

Merchant	Authorized merchant versus actual merchant.
Quantity	Authorized quantity versus actual quantity.
Maximum amount	Authorized maximum amount versus actual amount.
Add-ons	Authorized add-ons versus actual add-ons.
Authorization expiry	Current time versus authorization expiry.
Inventory	Current available inventory.
Price	Current merchant price.
Offer validity	Whether the applicable offer is valid.
Merchant availability	Whether the merchant is available where applicable.

Required examples include:

Plain Text

Authorized maximum = ₹70,000
 Actual transaction = ₹69,999
 → ALLOW if all other checks pass.

Authorized maximum = ₹70,000
 Actual transaction = ₹74,999
 → BLOCK.

Authorized product = LEN-001
 Actual product = LEN-002
 → BLOCK.

Authorized quantity = 1
 Actual quantity = 2
 → BLOCK.

Authorized add-ons = none
 Actual transaction contains an unauthorized warranty
 → BLOCK.

Authorization expired
 → BLOCK.

Current merchant state is ambiguous or requires human clarification
→ REVIEW.

A **REVIEW** decision must never automatically execute payment.

6. Mandatory Decisions

6.1 ALLOW

ALLOW means everything satisfies the authorization and the current merchant state. Only ALLOW may proceed to Razorpay Test Checkout.

6.2 REVIEW

REVIEW means something is ambiguous or requires human clarification. The interface must show the reason, conflicting information, evidence, and required action. The system must not initiate payment.

6.3 BLOCK

BLOCK means the transaction violates the authorization or a mandatory safety condition. The interface must show the exact reason, authorized value, actual value, evidence, authorization ID, and transaction ID. The system must not initiate payment.

Decision	Payment behavior	Required explanation
ALLOW	May proceed to Razorpay Test Checkout.	Show the successful checks and decision.
REVIEW	Must not initiate payment.	Show reason, conflict, evidence, and required human action.
BLOCK	Must not initiate payment.	Show exact reason, authorized value, actual value, evidence, authorization ID, and transaction ID.

7. Mandatory Frontend

The final product must be a real web application. It must be visibly branded:

Plain Text

It must not be a productivity dashboard. The mandatory screens are:

1. Dashboard.
2. New Intent / User Request.
3. AI Proposal.
4. Authorization.
5. Merchant State.
6. Transaction Verification.
7. ALLOW Result.
8. REVIEW Result.
9. BLOCK Result.
10. Audit Trail.

The Dashboard must show real data generated by the application rather than meaningless hardcoded counters. Useful dashboard concepts include transactions, allowed decisions, review decisions, blocked decisions, blocked value, recent decisions, and active agents. Any example or mock numbers from the source PRD are examples only and must not be hardcoded as permanent values.

8. Transaction Verification Screen

The Transaction Verification screen must show a clear three-way comparison.

Plain Text

AUTHORIZED

Product: LEN-001
Maximum: ₹70,000
Quantity: 1
Merchant: XYZ

CURRENT MERCHANT STATE

Product: LEN-001
Price: ₹69,999
Inventory: 3
Offer: ACTIVE

ACTUAL TRANSACTION

Product: LEN-001

Amount: ₹69,999

Quantity: 1

It must show individual checks for Product, Merchant, Quantity, Amount, Authorization, Current price, Inventory, and Offer.

Plain Text

- ✓ Product
- ✓ Merchant
- ✓ Quantity
- ✓ Amount
- ✓ Authorization
- ✓ Current price
- ✓ Inventory
- ✓ Offer

The screen must then show one of:

Plain Text

ALLOW

or

REVIEW

or

BLOCK

The explanation must be tied to actual stored evidence.

9. Audit Trail

Every important event must be recorded. The audit flow must include, as applicable:

Plain Text

Intent received

↓

Intent structured


```
↓
AI proposal generated
↓
User approved
↓
Authorization created
↓
Merchant state fetched
↓
Transaction received
↓
Authorization verified
↓
Decision made
↓
Payment allowed / blocked
```

The audit trail must allow a decision to be reconstructed from stored evidence. It must distinguish the intent, structured interpretation, proposal, user approval, authorization record, fetched merchant state, received transaction, verification, decision, and payment result.

10. Required Backend and Technology

The implementation must use:

Layer	Required technology or rule
Backend language	Python
API framework	FastAPI
Initial database	SQLite
Validation and schemas	Pydantic
Database access	SQLAlchemy or equivalent
Frontend	React + Vite
Initial LLM	Gemini API
LLM architecture	Isolate the provider so another model can be substituted later.
Payment	Razorpay Test Mode only

One clean FastAPI backend is preferred. The project must not build microservices.

11. Required Data Models

At minimum, the backend must define models for:

Plain Text

```
User
Agent
Merchant
Product
MerchantState
Intent
Authorization
Transaction
VerificationResult
AuditEvent
```

The first phase must prioritize the foundational models needed for backend creation of authorization and transaction records, along with the supporting entities and audit system.

12. Required API Areas

The backend must provide API areas for:

Plain Text

```
/intent
/authorization
/merchant
/merchant-state
/transaction
/verify
/audit
/payment
```

The initial phase must establish the API foundation and the endpoints required to create and inspect the core records. Later phases may complete the full workflow, verification

behavior, audit visualization, and Razorpay Test Mode integration, but the implementation must stop after each phase as directed in Section 16.

13. Security Architecture

The system must clearly demonstrate the difference between LLM reasoning and machine-enforceable financial authority.

The LLM can be wrong, manipulated, stale, or unavailable. IntentLock must still protect the transaction boundary. The control separation is:

Plain Text

LLM → interpretation/proposal

IntentLock → authorization + deterministic verification + decision

Razorpay → test payment execution

The LLM may interpret and propose, but it cannot authorize payment, alter the authorization contract, bypass a safety check, or execute payment. Only IntentLock may decide whether the transaction is allowed to proceed, and only an ALLOW decision may reach Razorpay Test Mode.

14. Required Demo Scenarios

The application must support at least the following scenarios.

Scenario 1 — Clean transaction

Plain Text

User:

"Buy this laptop under ₹70,000."

Approved:

₹69,999

Current merchant state:

₹69,999

→ ALLOW

→ Razorpay Test Checkout

Scenario 2 — Price escalation

Plain Text

Authorized:
₹70,000

Actual:
₹74,999

→ BLOCK

Scenario 3 — Unauthorized add-on

Plain Text

User authorized:
Laptop only

Actual:
Laptop + warranty

→ BLOCK

Scenario 4 — Ambiguous intent

Plain Text

User:
"Buy the usual laptop."

Multiple valid products exist.

→ REVIEW
→ Ask user to clarify
→ No payment

Scenario 5 — Stale or changed merchant state

Plain Text

User approved product at one price.

Merchant price changes before transaction.

- BLOCK or REVIEW according to the defined verification policy
- Explain the evidence

The final scenario deliberately leaves the exact BLOCK-versus-REVIEW result as a verification-policy decision; the application must define and document that policy rather than handle the condition silently.

15. Adversarial Testing

The test suite must include:

- Price escalation.
- Product substitution.
- Unauthorized add-on.
- Quantity manipulation.
- Expired offer.
- Inventory race or inventory change.
- Merchant change.
- Stale recommendation.
- Ambiguous intent.
- Duplicate transaction.
- Authorization expiry.
- Prompt injection inside merchant or product content.

The major security tests must work without requiring an LLM call. The deterministic backend must therefore be independently testable with structured intents, authorization contracts, merchant states, and actual transaction requests.

16. Technology Boundaries and Explicit Non-Goals

Do not build any of the following:

Prohibited product or capability	Boundary
Productivity application	Not the intended product.
Focus application	Not the intended product.
Habit tracker	Not the intended product.

Task manager	Not the intended product.
Generic chatbot	Not the intended product.
Generic shopping recommender	Not the intended product.
RL agent	Not required.
Custom ML model	Not required.
Fraud detection model	Not the product.
Blockchain	Not required.
Cryptocurrency system	Not required.
Payment gateway	IntentLock is a safety/control layer, not a gateway.
Microservice architecture	Use one clean FastAPI backend.
Production payment infrastructure	Use Razorpay Test Mode only.

This project is a prototype of a transaction safety and control layer.

17. Development Method and Phase Gates

The system must not be built all at once. It must be built phase by phase. After each phase, stop and show the result. Do not automatically jump to the next phase.

Phase	Scope	Required gate
Phase 1	Backend foundation + database models.	Stop and show the result.
Phase 2	Intent → authorization workflow.	Stop and show the result.
Phase 3	Deterministic transaction verification.	Stop and show the result.
Phase 4	Adversarial scenarios + tests.	Stop and show the result.
Phase 5	LLM semantic layer.	Stop and show the result.
Phase 6	Frontend.	Stop and show the result.
Phase 7	Razorpay Test Mode.	Stop and show the result.

Phase 8	Evaluation, audit visualization, polish, and deployment.	Stop and show the result.
---------	--	---------------------------

18. First Task — Phase 1 Only

Before writing feature code, the implementation agent must complete the following actions:

1. Inspect the current project.
2. Identify the incorrect “Personal Focus Console” implementation.
3. Do not continue developing that product.
4. Reorient the application to IntentLock.
5. Preserve any generic project setup that is useful.
6. Replace the incorrect product and UI implementation.
7. Build only Phase 1.

Phase 1 is **Backend foundation + database models**. It must not include the dashboard, payment integration, or later phases.

At the end of Phase 1, report all of the following:

Required report item	Expected content
Project structure	Relevant directories and files created or retained.
Technologies used	Frameworks, libraries, database, and other implementation choices.
Database schema	Tables, fields, relationships, and constraints created.
Models created	User, Agent, Merchant, Product, MerchantState, Intent, Authorization, Transaction, VerificationResult, and AuditEvent as applicable to Phase 1.
API endpoints	Endpoints implemented in the initial backend foundation.
How to run the application	Exact local setup and run commands.
Tests created	Test files and behaviors covered.

Test results

Test command, pass/fail result, and any known limitations.

Then stop. Do not build the dashboard, Razorpay integration, or other phases yet.

19. Final Non-Negotiable Requirement

If there is any ambiguity between an implementation interpretation and the Product Requirements Document, the PRD wins.

The final application must be immediately recognizable as:

Plain Text

INTENTLOCK
AI-Commerce Transaction Safety Gateway

It must not be recognizable as:

Plain Text

Personal Focus Console
Focus App
Productivity App
Intention Tracker

The current preview is a failed implementation and must be corrected before continuing.

Appendix A — Complete Source Preservation

The full second attachment is appended below verbatim so that every requirement, example, prohibition, phase gate, and correction instruction remains available in this standalone PRD.

IMPORTANT: STOP THE CURRENT BUILD AND CORRECT THE PROJECT.

The current preview is "PERSONAL FOCUS CONSOLE" with features such as "New focus", "Set an intention", etc.

THIS IS THE WRONG APPLICATION.

The project is NOT a productivity app, focus app, intention tracker, habit app, or task manager.

The uploaded Product Requirements Document:

"Product_Requirements_Document_IntentLock.docx"

is the source of truth.

The actual project is:

INTENTLOCK

AI-Commerce Transaction Safety Gateway

Razorpay Buildathon — Track 01: AI Growth & Agentic Commerce

Core principle:

THE AI PROPOSES.

THE USER AUTHORIZES.

INTENTLOCK ENFORCES.

RAZORPAY EXECUTES.

IntentLock is a safety/control layer between an AI commerce agent and payment execution.

=====

1. REQUIRED PRODUCT WORKFLOW

=====

The application must implement this flow:

USER

↓

Natural-language purchase request

↓

LLM interprets the request

↓

Structured intent

↓

AI purchase proposal

↓

USER APPROVES

↓

IntentLock creates an authorization contract

↓

Current merchant state is verified

↓

Actual transaction request arrives

↓

IntentLock deterministically verifies the transaction

↓

ALLOW / REVIEW / BLOCK

↓

ONLY ALLOW can proceed to Razorpay TEST MODE

↓

Payment result

↓

Complete audit trail

The LLM is ONLY a language/semantic layer.

The LLM must NEVER be the source of financial authority.

The LLM must NEVER directly authorize payment.

The LLM must NEVER directly execute payment.

The deterministic IntentLock backend must remain capable of making the major security decisions even if the LLM is unavailable.

=====

=====

2. WHAT INTENTLOCK MUST DO

A user might say:

"Buy me a laptop under ₹70,000."

The LLM may convert this into:

Action: PURCHASE

Category: LAPTOP

Maximum amount: ₹70,000

Quantity: 1

The AI may propose:

Product: Lenovo X

Price: ₹69,999

Quantity: 1

The user approves.

IntentLock creates a machine-enforceable authorization containing information such as:

- authorization ID
- user ID
- agent ID
- merchant ID
- action
- product/item
- quantity
- maximum amount
- currency
- allowed add-ons
- expiry
- confirmation requirement

Then IntentLock checks the CURRENT merchant state and the ACTUAL transaction.

=====

=====

3. GENERIC VERIFICATION — VERY IMPORTANT

Do NOT hardcode rules for Lenovo, laptops, or any specific product.

Lenovo is ONLY an example/demo product.

The verification engine must work generically using IDs and transaction fields.

It must compare:

AUTHORIZED

vs

CURRENT MERCHANT STATE

vs

ACTUAL TRANSACTION

Required checks include:

- authorized product vs actual product
- authorized merchant vs actual merchant
- authorized quantity vs actual quantity
- authorized maximum amount vs actual amount
- authorized add-ons vs actual add-ons
- authorization expiry
- current inventory
- current price
- offer validity
- merchant availability where applicable

Examples:

Authorized maximum = ₹70,000

Actual transaction = ₹69,999

→ ALLOW if all other checks pass.

Authorized maximum = ₹70,000

Actual transaction = ₹74,999

→ BLOCK.

Authorized product = LEN-001

Actual product = LEN-002

→ BLOCK.

Authorized quantity = 1

Actual quantity = 2

→ BLOCK.

Authorized add-ons = none

Actual transaction contains an unauthorized warranty

→ BLOCK.

Authorization expired

→ BLOCK.

Current merchant state is ambiguous or requires human clarification

→ REVIEW.

REVIEW must NEVER automatically execute payment.

=====

=====

4. MANDATORY DECISIONS

ALLOW

Everything satisfies the authorization and current merchant state.

Only ALLOW may proceed to Razorpay Test Checkout.

REVIEW

Something is ambiguous or requires human clarification.

Show:

- reason
- conflicting information
- evidence
- required action

Do NOT initiate payment.

BLOCK

The transaction violates authorization or a mandatory safety condition.

Show:

- exact reason
- authorized value
- actual value

- evidence
- authorization ID
- transaction ID

Do NOT initiate payment.

=====

=====

5. MANDATORY FRONTEND

The final product MUST be a real web application.

It must visibly be branded:

INTENTLOCK

Agentic Commerce Security

Do NOT create a productivity dashboard.

Mandatory screens:

1. Dashboard
2. New Intent / User Request
3. AI Proposal
4. Authorization
5. Merchant State
6. Transaction Verification
7. ALLOW Result
8. REVIEW Result
9. BLOCK Result
10. Audit Trail

The Dashboard should show real data generated by the application, not meaningless hardcoded counters.

Example dashboard concepts:

- transactions
- allowed

- review
- blocked
- blocked value
- recent decisions
- active agents

Example/mock numbers from the PRD are examples only.
DO NOT hardcode those numbers as permanent values.

=====

=====

6. TRANSACTION VERIFICATION SCREEN

Show a clear comparison:

AUTHORIZED

Product: LEN-001

Maximum: ₹70,000

Quantity: 1

Merchant: XYZ

CURRENT MERCHANT STATE

Product: LEN-001

Price: ₹69,999

Inventory: 3

Offer: ACTIVE

ACTUAL TRANSACTION

Product: LEN-001

Amount: ₹69,999

Quantity: 1

Then show individual checks:

✓ Product

- ✓ Merchant
- ✓ Quantity
- ✓ Amount
- ✓ Authorization
- ✓ Current price
- ✓ Inventory
- ✓ Offer

Then:

ALLOW

or

REVIEW

or

BLOCK

The explanation must be tied to actual stored evidence.

=====

=====

7. AUDIT TRAIL

Every important event must be recorded.

Example:

Intent received

↓

Intent structured

↓

AI proposal generated

↓

User approved

↓

Authorization created

↓

Merchant state fetched

↓

Transaction received

↓

Authorization verified

↓

Decision made

↓

Payment allowed / blocked

The audit trail must allow the decision to be reconstructed.

=====

=====

8. REQUIRED BACKEND

Use:

Python

FastAPI

SQLite initially

Pydantic

SQLAlchemy or equivalent

React + Vite frontend

Use Gemini API initially for LLM functionality, with the LLM provider isolated so another model can be substituted later.

Razorpay integration must use TEST MODE ONLY.

Do not build microservices.

One clean FastAPI backend is preferred.

=====

=====

9. REQUIRED DATA MODELS

- At minimum:
- User
 - Agent
 - Merchant
 - Product
 - MerchantState
 - Intent
 - Authorization
 - Transaction
 - VerificationResult
 - AuditEvent

=====

=====

10. REQUIRED API AREAS

- /intent
- /authorization
- /merchant
- /merchant-state
- /transaction
- /verify
- /audit
- /payment

=====

=====

11. IMPORTANT SECURITY ARCHITECTURE

The system must demonstrate the difference between:

LLM reasoning

and

machine-enforceable financial authority.

The LLM can be wrong, manipulated, stale, or unavailable.

IntentLock must still protect the transaction boundary.

Therefore:

LLM → interpretation/proposal

IntentLock → authorization + deterministic verification + decision

Razorpay → test payment execution

=====

=====

12. REQUIRED DEMO SCENARIOS

The application must support at least these scenarios:

SCENARIO 1 — CLEAN TRANSACTION

User:

"Buy this laptop under ₹70,000."

Approved:

₹69,999

Current merchant state:

₹69,999

→ ALLOW

→ Razorpay Test Checkout

SCENARIO 2 — PRICE ESCALATION

Authorized:

₹70,000

Actual:

₹74,999

→ BLOCK

SCENARIO 3 — UNAUTHORIZED ADD-ON

User authorized:

Laptop only

Actual:

Laptop + warranty

→ BLOCK

SCENARIO 4 — AMBIGUOUS INTENT

User:

"Buy the usual laptop."

Multiple valid products exist.

→ REVIEW

→ Ask user to clarify

→ No payment

SCENARIO 5 — STALE/CHANGED MERCHANT STATE

User approved product at one price.

Merchant price changes before transaction.

→ BLOCK or REVIEW according to the defined verification policy

→ Explain the evidence

=====

=====

13. ADVERSARIAL TESTING

Include tests for:

- price escalation
- product substitution
- unauthorized add-on

- quantity manipulation
- expired offer
- inventory race/change
- merchant change
- stale recommendation
- ambiguous intent
- duplicate transaction
- authorization expiry
- prompt injection inside merchant/product content

The major security tests must work without requiring an LLM call.

=====

=====

14. TECHNOLOGY BOUNDARIES

DO NOT build:

- productivity application
- focus application
- habit tracker
- task manager
- generic chatbot
- generic shopping recommender
- RL agent
- custom ML model
- fraud detection model
- blockchain
- cryptocurrency system
- payment gateway
- microservice architecture
- production payment infrastructure

This is a prototype of a transaction safety/control layer.

=====

=====

15. DEVELOPMENT METHOD

DO NOT attempt to build everything at once.

Build phase-by-phase.

PHASE 1:

Backend foundation + database models.

STOP and show me the result.

PHASE 2:

Intent → authorization workflow.

STOP and show me the result.

PHASE 3:

Deterministic transaction verification.

STOP and show me the result.

PHASE 4:

Adversarial scenarios + tests.

STOP and show me the result.

PHASE 5:

LLM semantic layer.

STOP and show me the result.

PHASE 6:

Frontend.

STOP and show me the result.

PHASE 7:

Razorpay Test Mode.

STOP and show me the result.

PHASE 8:

Evaluation, audit visualization, polish and deployment.

DO NOT automatically jump to the next phase.

=====

=====

16. FIRST TASK — DO THIS NOW

Before writing feature code:

1. Inspect the current project.
2. Identify the incorrect "Personal Focus Console" implementation.
3. Do NOT continue developing that product.
4. Reorient the application to IntentLock.
5. Preserve any generic project setup that is useful.
6. Replace the incorrect product/UI implementation.
7. Build ONLY Phase 1.

At the end of Phase 1, report:

- project structure
- technologies used
- database schema
- models created
- API endpoints
- how to run the application
- tests created
- test results

Then STOP.

Do not build the dashboard, payment integration, or other phases yet.

=====

=====

FINAL NON-NEGOTIABLE REQUIREMENT

If there is any ambiguity between your interpretation and the Product Requirements Document, the PRD wins.

The final application must be recognizable immediately as:

INTENTLOCK

AI-Commerce Transaction Safety Gateway

NOT:

Personal Focus Console

Focus App

Productivity App

Intention Tracker

The current preview is therefore considered a FAILED IMPLEMENTATION and must be corrected before continuing.